Zephyr Project

IT-Schulungen.com
Technology & Transformation

Logging aktivieren und filtern

3.02

Logging sichtbar steuern

START

Startzustand

native_sim bauen

printk-Ausgaben prüfen

Ausgangslage festhalten

MODUL

Logging-Modul

CONFIG_LOG aktivieren

LOG_MODULE_REGISTER

Level zuordnen

NACHWEIS

Filterwirkung

Info- und Debug-Build

.config vergleichen

Ergebnistabelle

Abnahme: sichtbare Meldungen entsprechen der wirksamen Level-Konfiguration.

Vom printk zum Logging-Subsystem

01	`printk()`	direkte Textausgabe ohne Modul- und Levelstruktur
02	`LOG_*`	strukturierte Meldung mit Modul, Level und Zeitinformation
03	`Aufgabentrennung`	Bedienantwort zur Shell, interne Diagnose ins Logging

Logging strukturiert Diagnose. Die Shell bleibt Bedienoberfläche.

Logging-Modul registrieren

TERMINAL

```
#include <zephyr/logging/log.h>
LOG_MODULE_REGISTER(event_app, LOG_LEVEL_INF);

LOG_INF("Event Service startet");
LOG_DBG("Zustand=%d", state);
```

01 **CONFIG_LOG=y**

aktiviert das Logging
und ein verfügbares Backend

02 **MODUL**

registriert den Modulnamen
und setzt den Compile-Time-Level

Jede Quelldatei verwendet ein klar benanntes Logging-Modul.

Schweregrade fachlich wählen

TERMINAL

```
LOG_ERR("Init");  LOG_WRN("Option");
LOG_INF("Service bereit");
LOG_DBG("Pegel=%d", level);
```

01 **ERR / WRN**

Fehler verhindert die Funktion
Warnung erlaubt eingeschränkten Betrieb

02 **INF / DBG**

Info beschreibt Meilensteine
Debug zeigt interne Details

Der Level beschreibt die fachliche Bedeutung einer Meldung.

Globale und modulspezifische Filterung

TERMINAL

```
CONFIG_LOG=y
CONFIG_LOG_DEFAULT_LEVEL=3

LOG_MODULE_REGISTER(event_app, LOG_LEVEL_DBG);
```

01 Compile-Time

Modul-Level bestimmt, welche Meldungen im Build verbleiben.

02 Runtime

Backendfilter können kompilierte Meldungen zusätzlich ausblenden.

03 Nachweis

.config und Konsolenausgabe gemeinsam prüfen.

Herausgefilterte Meldungen stehen später nicht zur Diagnose bereit.

Praxisauftrag: fünf Arbeitsschritte

Jeder Schritt liefert ein beobachtbares Ergebnis

01	02	03	04	05
Aktivieren	**Registrieren**	**Ersetzen**	**Vergleichen**	**Dokumentieren**
CONFIG_LOG=y ist wirksam	event_app erscheint als Modul	printk() wird zu LOG_*	Info- und Debug-Build unterscheiden sich	Soll/Ist-Tabelle ist vollständig

Lernkontrolle: Filterwirkung anhand einer konkreten Meldung erklären.

Abnahme: Filterwirkung nachweisen

INFO-BUILD

Sichtbar

ERR · WRN · INF

DEBUG-BUILD

Zusätzlich

DBG für event_app

PRÜFUNG

Zwei Belege

.config kontrollieren

Konsolenausgabe vergleichen

Ergebnis: Levelwahl, Build-Konfiguration und sichtbare Ausgabe stimmen überein.